

ChannelMamba: A Mamba-Driven Selective State-Space Model for Channel Prediction of High-Mobility MIMO in 6G IoT

Huagang Shi¹, Kaibo Jin¹, Xiaoquan Ren¹, Wei Li¹, and Yi Zhou¹, *Member, IEEE*

Abstract—Accurate channel state information (CSI) prediction is essential for 6G massive multiple-input multiple-output (m-MIMO) IoT systems. Deep learning models, such as Transformers, exhibit quadratic computational complexity, resulting in significant efficiency bottlenecks when processing high-dimensional, long sequences channel data in high-mobility scenarios. The Mamba architecture, distinguished by its unique selective state-space model (SSM), presents a promising solution which combines linear computational complexity with robust capabilities for modeling long-range dependencies. Building on this foundation, we propose ChannelMamba, an end-to-end model specifically designed for channel prediction. First, the model employs a dual-domain input module that captures comprehensive channel features by concurrently processing frequency-domain CSI and delay-domain channel impulse response (CIR) data. Subsequently, we develop a cross-path parameter-sharing strategy for the Mamba modules to efficiently capture temporal channel dynamics while enhancing model generalization. Furthermore, to address the multi-dimensional dependencies and global context inherent in channel data, we design a bidirectional Mamba module for cross-feature modeling, enhanced with a lightweight attention mechanism. Finally, extensive experimental evaluations across various standard scenarios demonstrate the significant advantages of ChannelMamba over baseline methods in terms of prediction accuracy, robustness, generalization and computational efficiency, achieving new state-of-the-art performance in channel prediction tasks.

Index Terms—Channel prediction, Mamba, massive multi-input multi-output (m-MIMO), orthogonal frequency division multiplexing (OFDM), deep learning.

Received 20 April 2025; revised 1 August 2025; accepted 28 September 2025. Date of publication 10 October 2025; date of current version 22 December 2025. This work was supported in part by the Natural Science Foundation of Henan Province under Grant 252300421231, in part by the National Natural Science Foundation of China under Grant 62303159 and Grant 62176088, in part by the International Strategic Innovative Project of National Key Research and Development Program of China under Grant 2023YFE0112500, in part by the Top Young Talent in Technological Innovation Program of Zhongyuan, in part by China Postdoctoral Science Foundation under Grant 2023M741008, and in part by the Program for Science and Technology Development of Henan Province under Grant 242102210195. The associate editor coordinating the review of this article and approving it for publication was H. Ye. (*Corresponding author: Yi Zhou.*)

Huagang Shi is with the School of Artificial Intelligence and Henan Engineering Research Center for Industrial Internet of Things, Henan University, Zhengzhou 450046, China, and also with the International Joint Research Laboratory for Cooperative Vehicular Networks of Henan, Zhengzhou 450046, China (e-mail: shihuagang@henu.edu.cn).

Kaibo Jin, Xiaoquan Ren, Wei Li, and Yi Zhou are with the School of Artificial Intelligence, Henan University, Zhengzhou 450046, China, and also with the International Joint Research Laboratory for Cooperative Vehicular Networks of Henan, Zhengzhou 450046, China (e-mail: jinkaibo@henu.edu.cn; renxiaoquan@henu.edu.cn; liwei@henu.edu.cn; zhoyui@henu.edu.cn).

Digital Object Identifier 10.1109/TWC.2025.3617502

I. INTRODUCTION

NEXT-GENERATION wireless systems (i.e., 6G) necessitate precise channel modeling and control to achieve demanding performance targets, including ultra-high data rates and ultra-low latency [1]. Multiple-input multiple-output (MIMO) technology serves as a crucial enabler, with its performance heavily reliant on accurate and timely channel state information (CSI) [2]. Consequently, the prediction of future channel states based on historical data has become essential for mitigating channel aging and maximizing system performance [3]. This task becomes particularly challenging in high-mobility scenarios, such as vehicular networks or drone communications, where rapid channel variations due to factors like Doppler shifts significantly accelerate channel aging, demanding highly adaptive prediction methods.

Deep learning (DL) approaches exhibit significant potential in addressing the wireless channel prediction problem [4]. In contrast to traditional statistical model-based methods [5], [6], DL models automatically learn complex and non-linear features from channel data, resulting in substantial improvements in prediction accuracy and robustness. Models that employ recurrent neural networks (RNNs) [7] effectively capture the time-series dependencies inherent in communication channels. However, conventional DL models may struggle to fully exploit the deep patterns embedded within complex wireless channel data [8]. Additionally, their inherently sequential processing characteristics limit the potential for training parallelism.

Recently, the Transformer architecture [9] has been widely adopted for MIMO channel prediction tasks. This adoption is primarily attributed to its self-attention mechanism, which excels at capturing long-range dependencies within sequences [10]. Jiang et al. [11] proposed a Transformer-based model that employs a parallel prediction scheme to effectively mitigate channel aging in high-mobility scenarios. Luan and Thompson [12] introduced the innovative Channelformer model, which continuously optimizes parameters based on real-time channel data. However, the core self-attention mechanism of the Transformer architecture exhibits $O(N^2)$ computational and memory complexity, where N represents the sequence length. This characteristic significantly restricts its ability to process long sequence data and limits its feasibility in high-dimensional applications, particularly when dealing with the extended history required to capture complex dynamics in mobile environments. For emerging 6G-IoT

applications demanding ultra-reliable low-latency communication, such as factory automation, this quadratic complexity poses a fundamental barrier to real-time prediction on resource-constrained edge devices. Furthermore, the practical deployment of models is impeded by their high inference times and training costs.

In light of the inherent limitations of the Transformer architecture, the research community is actively exploring more efficient paradigms for sequence modeling. While various Transformer variants, such as those utilizing sparse or linear attention mechanisms [13], aim to reduce computational complexity, they often compromise a degree of modeling performance. In this context, the Mamba architecture [14], [15], a novel sequence model based on S4 [16], presents distinct advantages. By employing a selective state update mechanism, the Mamba architecture achieves performance on par with or superior to that of Transformers on benchmarks, while reducing complexity to linear, i.e., $O(N)$. Studies indicate that Mamba excels in tasks such as text processing [17], image analysis [18], [19] and time series forecasting [20], [21], demonstrating superior efficiency and precision, particularly with long sequences and high-dimensional inputs. The TSCMamba model proposed by Ahamed and Cheng [22] integrates time-domain and frequency-domain features, significantly enhancing model expressiveness and classification accuracy for time series classification. The linear complexity inherent in the Mamba architecture can alleviate the computational requirements of massive MIMO systems and long-term channel modeling. Additionally, its foundation in selective state-space model (SSM) effectively captures sequential dynamics, which are crucial for time-varying channels, including those affected by high mobility. These attributes suggest significant potential for the Mamba architecture to address current bottlenecks in efficiency and accuracy in channel prediction.

Despite the demonstrated success and suitability of the Mamba architecture, its application to complex wireless channel prediction, particularly for MIMO systems, remains largely unexplored. The implementation of the Mamba architecture for MIMO channel prediction faces unique challenges. First, the high-dimensional (spatial, frequency, temporal) and multipath nature of MIMO orthogonal frequency division multiplexing (MIMO-OFDM) data necessitates models that can effectively capture intricate cross-dimensional dynamics. Second, wireless channels, especially under high mobility, exhibit both strong short-term correlations and complex long-range non-stationary patterns, which require robust memory capacity and accurate state representation from the sequence model.

Moving beyond conventional recurrent and attention-based sequence modeling paradigms, this paper introduces the Mamba architecture for MIMO-OFDM channel prediction. By leveraging the core advantages of the selective SSM foundation of the Mamba architecture, this work aims to address the performance and efficiency bottlenecks inherent in existing channel prediction methods, particularly those exacerbated by high mobility. Specifically, we propose Channel¹, an

end-to-end deep learning model designed for this task, which encompasses several key contributions summarized as follows:

- We propose a novel Mamba-based channel prediction methodology specifically designed for MIMO-OFDM systems operating in potentially high-mobility environments. To the best of our knowledge, this work represents the first adaptation of the Mamba architecture to channel prediction tasks.
- We design specialized Mamba configurations within the ChannelMamba model. Firstly, we strategically develop and optimize the cross-path parameter-sharing Mamba modules to efficiently capture the rapid temporal dynamics characteristic of high-mobility environments. Secondly, we design a bidirectional Mamba structure to explicitly model and disentangle the intricate cross-dimensional dependencies inherent in MIMO-OFDM channel data.
- We develop effective feature handling strategies within the ChannelMamba model by utilizing dual-domain input fusion for comprehensive channel representation. Additionally, we employ a specifically designed lightweight attention module for efficient global context integration, which enhances prediction performance in an effective manner.
- The proposed ChannelMamba model achieves new state-of-the-art (SOTA) performance in channel prediction tasks, significantly outperforming baseline methods as demonstrated through comprehensive experimental evaluations across multiple standard simulation scenarios.

The remainder of this paper is organized as follows. Section II reviews the related work. Section III presents the system model. Section IV presents the preliminaries. Section V proposes the ChannelMamba model. Section VI conducts experiments. Section VII draws conclusions.

II. RELATED WORK

In this section, we review the related work from the following two aspects: (1) Wireless channel prediction; (2) Mamba-based models.

A. Wireless Channel Prediction

Early research primarily relied on modeling assumptions concerning the statistical properties of communication channels. Common methods include Autoregressive models [5] and various forms of Kalman Filters [6]. Approaching from a physical propagation perspective, Gong et al. [23] developed an accurate near-field Line-of-Sight channel model specifically designed for MIMO communication systems. These approaches facilitate recursive prediction by explicitly modeling the channel state transition process. While they are computationally efficient and theoretically robust, their modeling capabilities and prediction accuracy encounter significant limitations in non-linear and highly dynamic practical wireless environments.

With the advancement of deep learning, data-driven methods have become increasingly prevalent. RNNs and their variants, such as long short-term memory (LSTM) networks [24] and

¹Our source code is available at <https://github.com/RxqJkb/ChannelMamba>

gated recurrent units (GRUs) [25], have been widely applied to channel prediction due to their capabilities in sequential modeling. Mourya et al. [26] utilized graph neural networks (GNNs) to model MIMO spatial correlations, integrating spectral domain information to enhance time-series predictions. The E2ENet proposed by Fan et al. [27] employs an end-to-end architecture that directly predicts the downlink channel from uplink pilots, offering a novel approach for channel prediction in massive antenna systems. To further enhance model robustness, recent works have demonstrated the benefits of fusing features from multiple domains. By jointly processing frequency-domain CSI and delay-domain channel impulse response (CIR), which represent the frequency selectivity and underlying physical multipath structure of the channel, respectively, models can learn more comprehensive and generalizable representations [28].

Although these methods [24], [25], [26], [27] effectively capture short-term dependencies, they often struggle to model long-range relationships within the temporal evolution of the channel. To address this limitation, the Transformer architecture has been introduced for channel prediction, leveraging its global attention mechanism and high parallelism. Kim et al. [29] applied the Transformer architecture to channel parameter estimation in Terahertz ultra-massive MIMO systems, demonstrating its capability to model complex spatial structures. Jin et al. [30] employed a Transformer-based approach that leverages obstacle information detected at the edge to enhance the accuracy of channel prediction. Furthermore, Zhou et al. [8] developed a Transformer-based channel prediction method that employs multi-head attention and positional encoding to enhance global information capture. Nevertheless, the Transformer architecture encounters challenges concerning inference efficiency and deployment feasibility. Its $O(N^2)$ attention complexity limits the processing of long sequences, and achieving robust generalization with non-stationary conditions remains a significant challenge. Although recent large pre-trained models (e.g., LLM4CP [28], WiFo [31]) demonstrate enhanced performance, their significant resource demands, such as large parameter sizes and high computational requirements, hinder deployment in resource-constrained environments.

In summary, while existing methods [8], [28], [29], [30], [31] have achieved notable progress in prediction accuracy and generalization, critical challenges persist concerning inference efficiency, the scalable modeling of long-term dependencies and deployment feasibility.

B. Mamba-Based Models

Recently, renewed interest in selective SSMs has led to the development of the Mamba architecture, an efficient sequence modeling architecture. The original Mamba architecture, proposed by Gu and Dao [14], employs an input-driven state update mechanism. This approach achieves modeling capabilities comparable to those of the Transformer architecture and has attained SOTA performance in tasks such as language modeling, audio processing and multivariate time series analysis. Subsequently, Dao and Gu [15] theoretically demonstrated the structural duality between the Mamba and Transformer

architectures, providing a theoretical foundation for selective SSMs to potentially replace self-attention mechanisms.

The Mamba architecture offers an alternative to the Transformer architecture and demonstrates remarkable adaptability with excellent performance across various domains. In natural language processing (NLP), the Hierarchical SSM design proposed by Bhirangi et al. [32] enables Mamba-based models to capture multi-scale dependencies in complex language sequences. Jiang et al. [33] developed Dual-Path Mamba architecture, which employs a dual-path fusion strategy to enhance the capture of both short- and long-term contexts, outperforming bidirectional Transformer structures in tasks such as speech separation. In computer vision, Vision Mamba [34] introduces bidirectional selective SSMs to achieve efficient visual representation learning without relying on self-attention mechanisms, yielding significant results in image classification and object detection.

In time series forecasting, the Mamba architecture has emerged as a significant alternative to the Transformer architecture. The S-Mamba model proposed by Wang et al. [35] outperforms both LSTM and Transformer models across multiple benchmark datasets. Li et al. [36] introduced CMMamba, which incorporates a channel mixing strategy to enhance the extraction of inter-variable dependencies within multivariate time series modeling. Models such as TimeMachine [37] and SiMBA-TS [21] combine Mamba modules with structural optimizations to improve long-term prediction stability. Notably, further research explores the integration of Mamba and Transformer architectures [31], [32], [38], [39], aiming to leverage their complementary strengths. ST-MambaSync [40] and the approach described in Mamba and Transformer [20] propose complementary structures specifically for traffic flow prediction and joint long-short term modeling, respectively.

In summary, the Mamba architecture has demonstrated robust capabilities in general sequence modeling, achieving performance that is comparable to or even superior to that of the Transformer architecture. Importantly, its $O(N)$ linear computational complexity directly addresses the efficiency and deployment challenges associated with the Transformer architecture. The proven success of the Mamba architecture in time series modeling encourages its exploration for channel prediction. However, the high-dimensional and physically-structured nature of MIMO channel data, with its coupled frequency, spatial and temporal domains, necessitates specialized architectural adaptations beyond the direct application of existing forecasting frameworks.

III. SYSTEM MODEL

A. Channel Representation

As illustrated in Fig. 1, a massive MIMO-OFDM downlink system is considered, operating at a carrier frequency f_c with K orthogonal subcarriers. The base station (BS) utilizes a uniform planar array (UPA) in the x-z plane, consisting of $N_x \times N_z = N_t$ transmit elements, with respective antenna spacings d_x and d_z . The UPA is capable of resolving the direction of departure (DoD), characterized by azimuth $\phi \in [-\pi, \pi)$ and elevation $\theta \in [-\pi/2, \pi/2)$. The transmit array steering vector

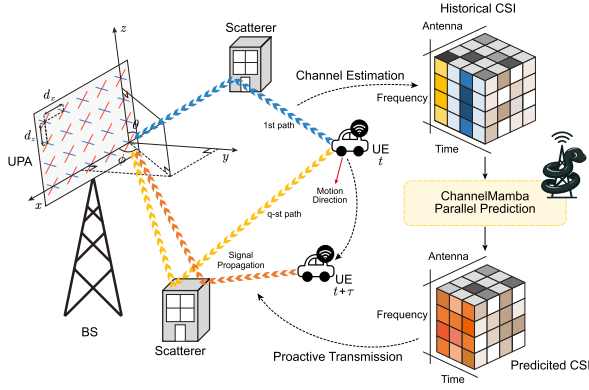


Fig. 1. MIMO-OFDM system model for channel prediction with moving UE.

$\mathbf{a}(\phi, \theta) \in \mathbb{C}^{N_t \times 1}$ represents the response to the direction (ϕ, θ) , with its m -th element given by

$$\mathbf{a}(\phi, \theta)_m = \frac{1}{\sqrt{N_t}} e^{j \frac{2\pi}{\lambda} ((m_x - 1)d_x \cos \phi \sin \theta + (m_z - 1)d_z \sin \phi \sin \theta)}, \quad (1)$$

where m linearizes the index (m_x, m_z) , and λ is the carrier wavelength. The user equipment (UE) is equipped with N_r receiving antennas, potentially forming its own array capable of resolving the direction of arrival (DoA).

Based on the geometric stochastic channel model (GSCM), whose principles are fundamentally described in 3GPP standards [38], the channel comprises Q discrete paths. The q -th path ($q = 1, 2, \dots, Q$) is characterized by a time-varying complex gain $g_q[t]$, a Doppler shift f_q , a delay τ_q , a DoD represented by the angles (ϕ_q, θ_q) , and a DoA represented by angles (ψ_q, ξ_q) . The channel matrix for the k -th subcarrier at time t , denoted as $\mathbf{H}_k[t] \in \mathbb{C}^{N_r \times N_t}$, is modeled as the sum of contributions from all paths

$$\mathbf{H}_k[t] = \sum_{q=1}^Q g_q[t] e^{j2\pi f_q t T_{\text{step}}} e^{-j2\pi f_k \tau_q} \mathbf{b}(\psi_q, \xi_q) \mathbf{a}^H(\phi_q, \theta_q), \quad (2)$$

where T_{step} represents the time step that accounts for the Doppler effect, while f_k denotes the frequency of the k -th subcarrier. $\mathbf{a}^H(\cdot)$ indicates the Hermitian transpose of the transmit steering vector, and $\mathbf{b}(\cdot)$ represents the receive array steering vector. The overall frequency-domain CSI tensor utilized in this paper is represented as $\mathbf{H}[t] \in \mathbb{C}^{K \times N_r \times N_t}$, which consolidates the channel matrices for all K subcarriers. Specifically, the slice corresponding to the k -th subcarrier is $\mathbf{H}_k[t]$. The movement of the UE over time leads to variations in the path parameters $(g_q, f_q, \tau_q, \phi_q, \theta_q, \psi_q, \xi_q)$.

Practical wideband channels exhibit a sparse delay-domain CIR, denoted as $\mathbf{h}[t] \in \mathbb{C}^{L_{\text{CIR}} \times N_r \times N_t}$, where L_{CIR} represents the maximum delay spread length, assuming wide-sense stationary uncorrelated scattering (WSSUS). The frequency-domain CSI $\mathbf{H}[t]$ is obtained through a K -point discrete Fourier transform (DFT) applied along the delay dimension for each antenna pair, expressed conceptually as

$$\mathbf{H}[t] = \mathcal{F}(\mathbf{h}[t]), \quad (3)$$

where $\mathcal{F}(\cdot)$ represents the appropriate DFT operation yielding the tensor $\mathbf{H}[t] \in \mathbb{C}^{K \times N_r \times N_t}$. While the frequency-domain

CSI $\mathbf{H}[t]$ is the primary focus for system operation, we also consider its corresponding delay-domain CIR $\mathbf{h}[t]$. The CIR provides a sparse, physically-grounded representation of the multipath environment, which is essential for improving the robustness of the channel model.

Pilot-based estimation yields noisy CSI $\tilde{\mathbf{H}}[t]$, which is modeled as

$$\tilde{\mathbf{H}}[t] = \mathbf{H}[t] + \mathbf{W}[t], \quad (4)$$

where $\mathbf{W}[t] \in \mathbb{C}^{K \times N_r \times N_t}$ represents additive white Gaussian noise (AWGN), its elements are independent and identically distributed (i.i.d.), following a complex Gaussian distribution denoted as $\mathcal{CN}(0, \sigma_w^2)$. The noise power σ_w^2 encompasses both thermal noise and quantization errors.

B. Problem Formulation

Given a sequence of channel states over a historical observation window of P time steps, denoted as $\mathbf{X} = \{\mathbf{H}[t - P + 1], \mathbf{H}[t - P + 2], \dots, \mathbf{H}[t]\}$, the channel prediction task aims to establish a mapping function $f_\theta: \mathbb{C}^{P \times K \times N_r \times N_t} \rightarrow \mathbb{C}^{L \times K \times N_r \times N_t}$, where θ represents the trainable parameters of the model. The objective is to predict the channel states for the next L future time steps, denoted as $\mathbf{Y} = \{\mathbf{H}[t + 1], \mathbf{H}[t + 2], \dots, \mathbf{H}[t + L]\}$. The model is trained by minimizing a loss function that quantifies the discrepancy between the true future channel states $\mathbf{Y}$ and the predicted states $\hat{\mathbf{Y}}$. This paper employs the normalized mean squared error (NMSE) as the objective function.

The model parameters θ are optimized by solving the following problem formulation

$$\begin{aligned} \min_{\theta} \quad & \mathbb{E} \left[\frac{\sum_{i=1}^L \|\mathbf{H}[t + i] - \hat{\mathbf{H}}[t + i]\|_F^2}{\sum_{i=1}^L \|\mathbf{H}[t + i]\|_F^2} \right] \\ \text{s.t.} \quad & \hat{\mathbf{Y}} = \{\hat{\mathbf{H}}[t + 1], \dots, \hat{\mathbf{H}}[t + L]\} = f_\theta(\mathbf{X}), \end{aligned} \quad (5)$$

where $\mathbb{E}[\cdot]$ denotes the expectation taken over the joint distribution of historical observations $\mathbf{X}$ and true future states $\mathbf{Y}$. The model $f_\theta(\cdot)$ maps $\mathbf{X}$ to the predicted future states $\hat{\mathbf{Y}}$, and $\|\cdot\|_F^2$ represents the squared Frobenius norm.

IV. PRELIMINARIES

A. State Space Models

SSMs offer a well-established framework for modeling sequential data, particularly well-suited for describing dynamic systems with latent states. As illustrated in Fig. 2(a), the classic linear time-invariant (LTI) SSM is defined by

$$\mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \mathbf{B}\mathbf{x}_t, \quad (6)$$

$$\mathbf{y}_t = \mathbf{C}\mathbf{h}_t, \quad (7)$$

where $\mathbf{x}_t$, $\mathbf{h}_t$ and $\mathbf{y}_t$ represent the input, latent state and output at time step t , respectively. The system matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ are constant parameters with the LTI assumption.

Continuous-time SSMs are typically discretized using a step size Δ . A common discrete-time representation is given by

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{x}_t, \quad (8)$$

$$\mathbf{y}_t = \bar{\mathbf{C}}\mathbf{h}_t, \quad (9)$$

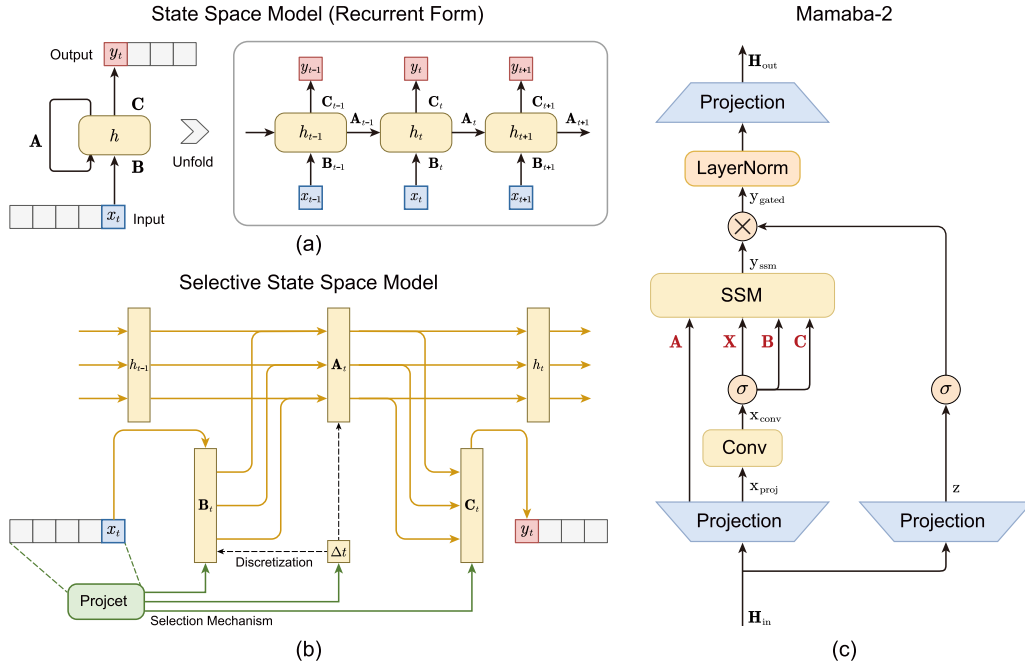


Fig. 2. Mamba architecture foundations. (a) Standard SSM. (b) Selective SSM Model. (c) Mamba-2 block.

where $\bar{\mathbf{A}}$, $\bar{\mathbf{B}}$ and $\bar{\mathbf{C}}$ represent the discretized system matrices. Building upon this formulation, Mamba [14] introduces the Selective SSM, as illustrated in Fig. 2(b). This model replaces static system parameters with those dynamically generated from the input $\mathbf{x}_t$. In this architecture, parameters such as the step size Δ_t and the matrices $\mathbf{B}_t$ and $\mathbf{C}_t$ are dynamically generated based on the input $\mathbf{x}_t$ through a selection mechanism. This input-dependent selectivity builds on the foundations of S4 [16] and HiPPO [41] which define a structured state matrix $\mathbf{A}$ capable of compressing long-range dependencies. By making $\mathbf{B}_t$ and $\mathbf{C}_t$ functions of the input, the model can selectively control the flow of information into and out of the latent state, thereby enabling it to efficiently model complex, non-stationary sequences. The specifics of this selection mechanism are detailed in the Mamba block description below.

B. Mamba Block

The Mamba block serves as the fundamental building unit of the Mamba architecture, as illustrated in Fig. 2(c). It mitigates the quadratic complexity ($O(N^2)$) of Transformers by substituting self-attention with a linear-time state-space model. The Mamba block processes sequences through a recurrent state update, where the computation at each step is independent of the sequence length, resulting in an overall linear complexity of $O(N)$. This paper employs the Mamba-2 variant as the primary component for the proposed ChannelMamba model.

First, the input $\mathbf{H}_{in}$ is processed through two distinct linear projection layers, resulting in $\mathbf{x}_{proj}$ and $\mathbf{z}$. These projections are intended for the subsequent convolutional/SSM path and the gating path, respectively.

For the projection $\mathbf{x}_{proj}$, it is processed through a 1D causal convolution layer (Conv1D) with a kernel size of d_{conv} to capture local contextual information. Following this, the

sigmoid linear unit (SiLU) activation function is applied, which is expressed as

$$\mathbf{x}_{conv} = \sigma(\text{Conv1D}(\mathbf{x}_{proj})), \quad (10)$$

where $\sigma(\cdot)$ denotes the sigmoid linear unit activation function, and $\text{Conv1D}(\cdot)$ represents the 1D causal convolution operation.

The resulting $\mathbf{x}_{conv}$ drives the core selective SSM mechanism. Based on $\mathbf{x}_{conv}$ and learned parameters, the selective SSM computes its output $\mathbf{y}_{ssm}$. Key parameters governing the dynamics of the selective SSM, such as the discretization step Δ and the matrices $\mathbf{B}$ and $\mathbf{C}$, are typically derived dynamically from $\mathbf{x}_{conv}$ through linear layers and activation functions, while $\mathbf{A}$ is often a structured matrix that is initialized appropriately (as in S4).

The other projection, $\mathbf{z}$, is also subjected to a SiLU activation. It is then multiplied element-wise with the selective SSM output $\mathbf{y}_{ssm}$ to perform gating, thereby modulating the flow of information. This operation is defined as

$$\mathbf{y}_{gated} = \mathbf{y}_{ssm} \odot \sigma(\mathbf{z}), \quad (11)$$

where $\mathbf{y}_{gated}$ represents the gated output and $\odot$ denotes the element-wise multiplication.

Finally, the gated output $\mathbf{y}_{gated}$ is first stabilized by a Layer Normalization layer and subsequently mapped back to the original hidden dimension through a linear projection, yielding the final output of the Mamba block, denoted as $\mathbf{H}_{out}$.

V. THE CHANNELMAMBA MODEL

As illustrated in Fig. 3, we propose and detail the ChannelMamba model, a Mamba-based approach specifically designed for MIMO-OFDM channel prediction.

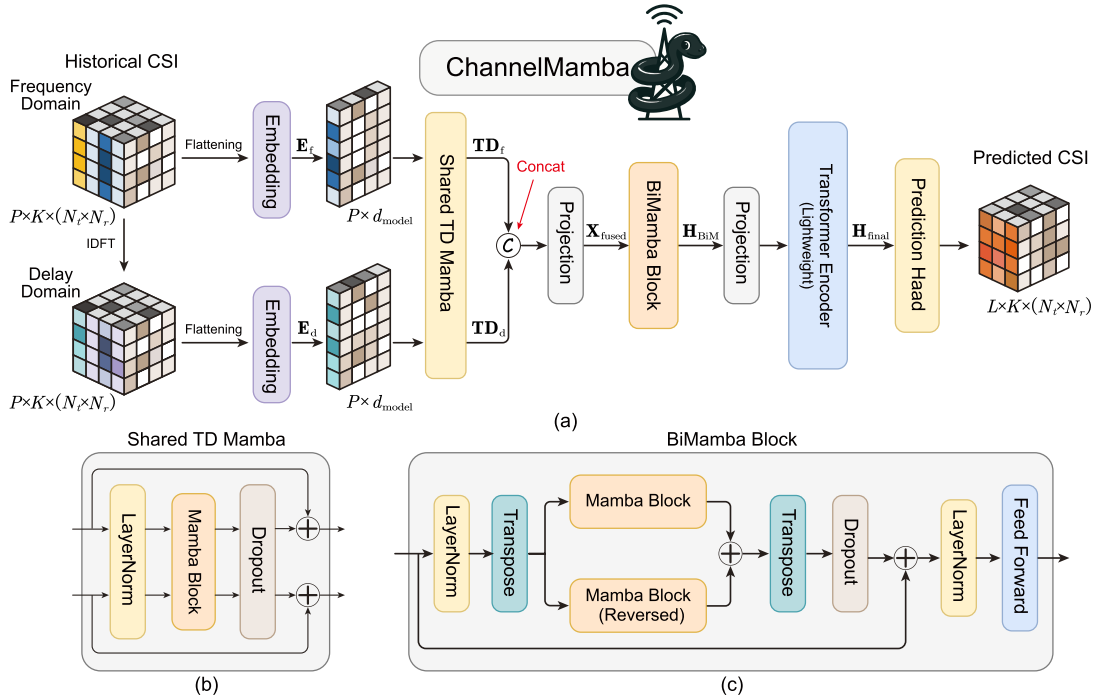


Fig. 3. Architecture of the ChannelMamba model. (a) Overall framework. (b) Shared TD Mamba block and (c) BiMamba block.

A. Dual-Domain Input Embedding

To leverage the sparse multipath structure inherent in the dense CSI, the ChannelMamba model internally generates the corresponding CIR using an inverse discrete Fourier transform (IDFT). By processing both domains simultaneously, the model exploits the physical structure of the CIR to enhance the robustness and generalization of its predictions. This module receives the historical frequency-domain CSI sequence $\mathbf{X} \in \mathbb{C}^{P \times K \times N_r \times N_t}$ as input, where P is the number of historical time steps.

First, the input complex frequency-domain CSI $\mathbf{H}[t'] \in \mathbb{C}^{K \times N_r \times N_t}$ at each time step $t' \in \{t - P + 1, t - P + 2, \dots, t\}$ within the historical window is converted into a real-valued representation. This process yields $\mathbf{X}_f[t'] \in \mathbb{R}^{K \times N_r \times N_t \times 2}$, typically achieved by concatenating the real and imaginary parts along a new dimension. Concurrently, to utilize delay-domain information, the corresponding time-domain CIR $\mathbf{h}[t'] \in \mathbb{C}^{L_{\text{CIR}} \times N_r \times N_t}$ is generated. This involves applying a K -point IDFT along the subcarrier dimension (K) to the complex CSI $\mathbf{H}[t']$. Assuming the length of the CIR equals the number of subcarriers ($L_{\text{CIR}} = K$, which may require zero-padding or truncation if $L_{\text{CIR}} \neq K$), the resulting CIR $\mathbf{h}[t']$ is also converted into a real-valued representation $\mathbf{X}_d[t'] \in \mathbb{R}^{L_{\text{CIR}} \times N_r \times N_t \times 2}$.

For each time step t' , the multi-dimensional real-valued CSI tensors, specifically the frequency-domain tensor $\mathbf{X}_f[t']$ and the delay-domain tensor $\mathbf{X}_d[t']$, are flattened along the subcarrier (K), receiving antenna (N_r), and transmitting antenna (N_t) dimensions. These flattened vectors, each with a dimension of $(K \times N_r \times N_t \times 2)$, are then concatenated across all P historical time steps. This process generates two sequences that are suitable for sequence modeling: $\mathbf{x}_f \in \mathbb{R}^{P \times (K \times N_r \times N_t \times 2)}$ and $\mathbf{x}_d \in \mathbb{R}^{P \times (K \times N_r \times N_t \times 2)}$.

Next, independent embedding layers are utilized. This approach enables the model to learn and differentiate domain-specific feature representations at an early stage. Two distinct linear embedding layers process the frequency-domain sequence $\mathbf{x}_f$ and the delay-domain sequence $\mathbf{x}_d$, respectively. Each layer maps the input features of dimension $(K \times N_r \times N_t \times 2)$ to the core hidden dimension of model, denoted as d_{model} . Subsequently, dropout is applied for regularization, resulting in the final embedded sequences $\mathbf{E}_f \in \mathbb{R}^{P \times d_{\text{model}}}$ and $\mathbf{E}_d \in \mathbb{R}^{P \times d_{\text{model}}}$.

B. Mamba Processing Core

The Mamba processing core is designed to model the complex structure of MIMO channel data, diverging from standard temporal forecasting applications in its architecture. It employs a cross-path parameter-sharing strategy for the temporal Mamba blocks to jointly process dual-domain inputs and subsequently applies a bidirectional Mamba structure across the feature dimension to capture spatial-frequency dependencies.

The embedded features $\mathbf{E}_f$ and $\mathbf{E}_d$ are first processed by a shared Layer Normalization layer, resulting in normalized features. These normalized sequences are then processed sequentially through a stack of M_{TD} shared Mamba blocks, which are specifically designed to capture the temporal dynamics (TD) of the channels, is illustrated in Fig. 3(b). This stack follows a pre-normalization design. Specifically, for the m -th block (where $m \in \{1, 2, \dots, M_{\text{TD}}\}$), the input is first passed through a Mamba layer, followed by a Dropout layer for regularization. A residual connection then adds the output of the Dropout layer to the input of the m -th block. The resulting output of the m -th block serves as the input for the $(m+1)$ -th block, thereby creating a deep feature extraction hierarchy.

Crucially, in the ChannelMamba model, the parameters of each corresponding m -th block (including internal weights and Layer Normalization parameters, if applied per block) are shared across both the frequency and delay-domain paths. This means that the same m -th Mamba block instance processes features from both paths sequentially. This parameter-sharing strategy compels the model to learn a unified representation of temporal dynamics from both domains, thereby enhancing generalization and reducing the overall parameter count. Based on the assumption that the underlying dynamics of channel evolution are consistent across different domains, this approach encourages the model to learn more general and robust temporal features. After processing through the M_{TD} shared Mamba blocks, the features encoding temporal dependencies are obtained: $\mathbf{TD}_f \in \mathbb{R}^{P \times d_{\text{model}}}$ for the frequency domain and $\mathbf{TD}_d \in \mathbb{R}^{P \times d_{\text{model}}}$ for the delay-domain. To efficiently capture the rapid temporal dynamics characteristic of high-mobility channels, we employ these shared Mamba blocks under a tailored parameter-sharing scheme.

To integrate information from the dual paths, $\mathbf{TD}_f$ and $\mathbf{TD}_d$ are concatenated along the feature dimension. A subsequent linear projection layer maps the concatenated features from a dimension of $2 \times d_{\text{model}}$ back to d_{model} . This process yields the fused feature $\mathbf{X}_{\text{fused}} \in \mathbb{R}^{P \times d_{\text{model}}}$.

To model complex dependencies within the feature dimension (d_{model}) of the fused feature $\mathbf{X}_{\text{fused}}$, which integrates spatial and frequency/delay information, we draw inspiration from [33], which utilizes the Mamba architecture for inter-variable correlations. To explicitly model the intricate cross-dimensional dependencies inherent in MIMO-OFDM channel data, we design a bidirectional Mamba structure, as detailed in Fig. 3(c). This module operates along the feature dimension rather than the time dimension, enabling it to capture the complex correlations among the embedded spatial, frequency and domain features. This operation is formally expressed as

$$\mathbf{H}_{\text{BiM}} = \text{BiMamba}(\mathbf{X}_{\text{fused}}), \quad (12)$$

where $\text{BiMamba}(\cdot)$ represents the Bidirectional Mamba structure. Initially, $\mathbf{X}_{\text{fused}}$ undergoes Layer Normalization and is subsequently transposed to dimensions $(P \times d_{\text{model}})$. This transposed feature is processed by independent forward and reverse Mamba blocks. The forward block captures dependencies from the first feature dimension to the last, while the reverse block captures dependencies from the last feature dimension back to the first. The outputs of these blocks are summed element-wise to produce an intermediate result. This result is then transposed back to dimensions $(P \times d_{\text{model}})$, regularized through Dropout, and combined with the original $\mathbf{X}_{\text{fused}}$ through a residual connection. The output of this residual connection is subsequently passed through Layer Normalization, followed by shared feed-forward network (FFN) blocks, denoted as L_{FFN} . Each FFN block applies standard non-linear transformations to further enhance the representational capabilities of the model. The final output of this stage is $\mathbf{H}_{\text{BiM}} \in \mathbb{R}^{P \times d_{\text{model}}}$.

While the Mamba backbone excels at capturing the causal, temporal evolution of the channel, certain dynamics in high-mobility MIMO systems are inherently non-local. For

example, the sudden blockage of a line-of-sight path can create an abrupt change across the entire observation window. To address this concern, we introduce a lightweight attention module designed to perform a final global context refinement. This module first significantly reduces the feature dimension from d_{model} to a smaller dimension d_{tf} ($d_{\text{tf}} \ll d_{\text{model}}$) using a linear layer applied to the output of the FFN, denoted as $\mathbf{H}_{\text{BiM}}$. Subsequently, a limited number of standard Transformer Encoder layers, denoted as N_{Enc} , are applied in this low-dimensional space. Each Transformer Encoder layer primarily consists of two sub-layers: multi-head attention and an FFN, which facilitate global feature interaction and refinement. Since all attention calculations and internal FFN operations occur in the low dimension d_{tf} , and given that the number of Transformer Encoder layers N_{Enc} is small, the computational complexity of this module (approximately $O(P^2 d_{\text{tf}})$). This complexity is significantly lower than that of applying standard Transformer layers in the original d_{model} dimension, which is $O(P^2 d_{\text{model}})$. After passing through the N_{Enc} Transformer Encoder layers, the final enhanced feature representation, denoted as $\mathbf{H}_{\text{final}} \in \mathbb{R}^{P \times d_{\text{tf}}}$, is obtained.

C. Prediction Head

The Prediction Head module of the ChannelMamba model is responsible for transforming the final extracted deep features $\mathbf{H}_{\text{final}} \in \mathbb{R}^{P \times d_{\text{tf}}}$ into channel predictions for the next L future time steps.

First, Layer Normalization is applied to $\mathbf{H}_{\text{final}}$. Next, a linear layer projects the feature dimension from d_{tf} back to the original flattened dimension $(K \times N_r \times N_t \times 2)$, resulting in the output feature matrix $\mathbf{O}$. This process is expressed as

$$\mathbf{O} = \text{Linear}(\text{LayerNorm}(\mathbf{H}_{\text{final}})), \quad (13)$$

where $\text{Linear}(\cdot)$ denotes the linear layer operation and $\mathbf{O} \in \mathbb{R}^{P \times (K \times N_r \times N_t \times 2)}$. We denote the target flattened dimension per time step as $d_{\text{out}} = K \times N_r \times N_t \times 2$.

To obtain the L -step prediction, the time dimension must be mapped from P to L . This is accomplished by first transposing the tensor $\mathbf{O}$ to dimensions $((K \times N_r \times N_t \times 2) \times P)$, applying an additional linear layer (which effectively maps the dimension P to L), and finally transposing it back. The operation is represented as

$$\hat{\mathbf{X}} = \text{Transpose}(\text{Linear}(\text{Transpose}(\mathbf{O}))), \quad (14)$$

where $\hat{\mathbf{X}} \in \mathbb{R}^{L \times (K \times N_r \times N_t \times 2)}$ represents the intermediate sequence of predictions generated by the model for the next L time steps in a flattened and real-valued format. $\text{Transpose}(\cdot)$ denotes the transpose operation.

Next, an inverse reshape operation is applied to $\hat{\mathbf{X}}$ to restore the original dimensions corresponding to subcarriers, receiving antennas and transmitting antennas, resulting in a tensor with the shape $(L \times K \times N_r \times N_t \times 2)$. The real and imaginary parts at each position (along the last dimension) are then combined to convert the representation into complex form. This process yields the final predicted output sequence

$$\hat{\mathbf{Y}} = \{\hat{\mathbf{H}}[t+1], \hat{\mathbf{H}}[t+2], \dots, \hat{\mathbf{H}}[t+L]\} \in \mathbb{C}^{L \times K \times N_r \times N_t}. \quad (15)$$

Algorithm 1 ChannelMamba Prediction Algorithm

-
- 1: **Input:** Historical CSI sequence $\mathbf{X}$
 - 2: **Output:** Predicted future CSI sequence $\hat{\mathbf{Y}}$
 - 3: $\triangleright$ **Dual-Domain Input Embedding**
 - 4: Convert $\mathbf{X}$ to real-valued frequency-domain sequence $\mathbf{x}_f$;
 - 5: Compute CIR sequence $\mathbf{h}$ through IDFT from $\mathbf{X}$ and convert to $\mathbf{x}_d$;
 - 6: Apply embedding layers with dropout to $\mathbf{x}_f$ and $\mathbf{x}_d$ yielding $\mathbf{E}_f$ and $\mathbf{E}_d$;
 - 7: $\triangleright$ **Shared TD Mamba and Fusion**
 - 8: Apply shared Layer Normalization to $\mathbf{E}_f$ and $\mathbf{E}_d$;
 - 9: Process results through M_{TD} shared Mamba blocks to obtain $\mathbf{TD}_f$ and $\mathbf{TD}_d$;
 - 10: Concatenate $\mathbf{TD}_f$ and $\mathbf{TD}_d$ then apply linear projection to obtain $\mathbf{X}_{fused}$;
 - 11: $\triangleright$ **BiMamba Block**
 - 12: Apply Layer Normalization and Transpose to $\mathbf{X}_{fused}$;
 - 13: Process with parallel Forward and Reverse Mamba blocks, then sum their outputs;
 - 14: Transpose the summed result back, apply Dropout, and add residual connection with $\mathbf{X}_{fused}$;
 - 15: Apply Layer Normalization and process through L_{FFN} FFN blocks to obtain $\mathbf{H}_{BiM}$;
 - 16: $\triangleright$ **Lightweight Transformer Encoder**
 - 17: Obtain reduced features from $\mathbf{H}_{BiM}$ through linear projection, which will serve as the initial input for the encoder layers;
 - 18: **for** $i = 1$ to N_{Enc} **do**
 - 19: Process current features through the i -th Transformer Encoder layer;
 - 20: **end for**
 - 21: $\triangleright$ **Prediction Head**
 - 22: Apply Layer Normalization to $\mathbf{H}_{final}$;
 - 23: Project features according to Eq. (13) to obtain $\mathbf{O}$;
 - 24: Map time dimension according to Eq. (14) to obtain $\hat{\mathbf{X}}$;
 - 25: Reshape and convert $\hat{\mathbf{X}}$ to complex $\hat{\mathbf{Y}}$ according to Eq. (15).
-

D. The ChannelMamba Prediction Algorithm

The ChannelMamba prediction algorithm is provided in Algorithm 1 and described as follows.

Dual-Domain Input Embedding (Lines 3–6): Initially, the model generates dual-domain representations by transforming the input $\mathbf{X}$ into the sequences $\mathbf{x}_f$ and $\mathbf{x}_d$ (via IDFT). Subsequently, these sequences are processed through independent embedding layers with dropout, resulting in the initial embeddings $\mathbf{E}_f$ and $\mathbf{E}_d$.

Shared TD Mamba and Fusion (Lines 7–10): The embeddings $\mathbf{E}_f$ and $\mathbf{E}_d$ first undergo shared Layer Normalization. They are then sequentially processed by M_{TD} shared Mamba blocks. The resulting temporal features ($\mathbf{TD}_f$ and $\mathbf{TD}_d$) are subsequently concatenated and linearly projected to obtain the fused features $\mathbf{X}_{fused}$.

BiMamba Block Processing (Lines 11–15): The BiMamba block processes the $\mathbf{X}_{fused}$. This stage involves applying Layer Normalization and transposition, processing the data with

parallel forward and reverse Mamba blocks, followed by summation. The result is then transposed back, after which dropout is applied, and a residual connection is added. Finally, the outcome is passed through Layer Normalization and L_{FFN} FFN blocks to yield $\mathbf{H}_{BiM}$.

Lightweight Transformer Encoding (Lines 16–20): Next, the lightweight attention module processes $\mathbf{H}_{BiM}$. The features are initially linearly projected to a reduced dimension and subsequently processed sequentially through N_{Enc} Transformer Encoder layers, yielding the final enhanced features $\mathbf{H}_{final}$.

Prediction Head (Lines 21–25): Finally, the Prediction Head transforms $\mathbf{H}_{final}$ into $\hat{\mathbf{Y}}$. This module applies Layer Normalization, projects features according to Eq. (13), maps the time dimension according to Eq. (14), and performs reshaping along with complex conversion according to Eq. (15).

E. Complexity Analysis

In this subsection, we evaluate the computational complexity of the proposed ChannelMamba model, focusing primarily on the number of multiplication operations required during online inference, since offline training does not impact deployment overhead.

The computational complexity of ChannelMamba arises from its key modules. The dual-domain input embedding requires $O(P \times d_{in} \times d_{model})$ operations, where the input dimension $d_{in} = K \times N_r \times N_t \times 2$ is substantial in MIMO systems. The subsequent M_{TD} shared TD Mamba blocks process temporal sequences, with their complexity dominated by internal projections, resulting in a cost of $O(M_{TD} \times P \times d_{model}^2)$. The BiMamba module, which contains M_{Bi} internal Mamba layers scanning along the feature dimension d_{model} , exhibits a complexity of $O(M_{Bi} \times d_{model} \times P^2)$ due to its internal structure where the effective sequence length processed by Mamba is d_{model} after transposition. The lightweight attention mechanism operates in a reduced dimension $d_{tf} \ll d_{model}$, with N_{Enc} layers contributing $O(N_{Enc} \times (P^2 \times d_{tf} + P \times d_{tf}^2))$ from self-attention and internal FFNs. The prediction head projects features from d_{tf} back to the output dimension, involving a complexity of approximately $O(P \times d_{tf} \times d_{out} + d_{out} \times P \times L)$.

In comparison, standard Transformer models typically involve complexities of $O(P^2 \times d_{model})$ from self-attention and $O(P \times d_{model}^2)$ from FFNs operating directly on the d_{model} dimension. By employing Mamba blocks with linear scaling characteristics for core temporal sequence modeling, and confining quadratic-cost operations to either a reduced dimension or across the feature dimension, ChannelMamba demonstrates a distinct and more efficient complexity profile compared to standard Transformer architectures.

VI. EXPERIMENTAL RESULTS AND ANALYSIS*A. Experimental Setup*

Experiments are conducted using 4 Intel Xeon Platinum 8358P CPUs and 4 NVIDIA GeForce RTX4090 GPUs, with model training implemented in PyTorch 2.1.1. Channel datasets compliant with the 3rd generation partnership project (3GPP) standards [38], [39] are generated using the

quasi deterministic radio channel generator (QuaDRiGa) simulator [42], considering the urban macro non-line-of-sight (UMa-NLoS) and urban micro non-line-of-sight (UMi-NLoS) channel models. The training primarily utilizes the UMa-NLoS scenario. Key system parameters include a carrier frequency of $f_c = 2.4$ GHz, an effective downlink bandwidth of $B = 8.64$ MHz (with $K = 48$ subcarriers, derived from a total simulated range of 17.28 MHz), and a BS equipped with a 4×4 dual-polarized UPA, resulting in $N_t = 32$ transmit ports (with $d = \lambda/2$). The UE is equipped with an omni-directional antenna. The channel is assumed to be quasi-static over the $P + L = 20$ consecutive frames considered for prediction ($P = 16$ historical steps and $L = 4$ future steps), with a sampling interval of $T_{\text{step}} = 0.5$ ms. UE mobility follows a linear trajectory. For the generation of training and validation data, UE speed v is sampled from a uniform distribution $U[10, 100]$ km/h, and the signal-to-noise ratio (SNR) is sampled from $[0, 30]$ dB. AWGN $\mathbf{W}[t]$ is used to simulate estimation errors. The datasets include 10,000 training segments and 1,000 validation segments. A single model is trained for each duplex mode and subsequently evaluated across all test conditions without retraining. The test dataset contains 100 segments for each of 10 discrete speeds, totaling 1,000 segments. Performance is assessed at these discrete speeds and specific SNR values during testing.

Model performance is evaluated using two key metrics: NMSE and spectral efficiency (SE).

- **NMSE:** As shown in Eq. (5), this metric directly quantifies the accuracy of the predicted CSI relative to the true future CSI. Lower NMSE values indicate higher prediction accuracy.
- **SE:** SE reflects the potential data throughput achievable when the predicted CSI is utilized for system operations. To realistically assess this, we calculate it for a single-stream transmission where the precoding vector $\mathbf{f}[t]$ is the dominant right singular vector of the predicted channel matrix $\hat{\mathbf{H}}[t]$. The performance is then evaluated using the true channel vector $\mathbf{h}[t]$, and the SE is expressed as

$$\text{SE} = \mathbb{E} [\log_2 (1 + \text{SNR} \cdot |\mathbf{h}[t]\mathbf{f}[t]|^2)], \quad (16)$$

where SNR denotes the average signal-to-noise ratio. This metric serves as an indirect measure of the utility of prediction for enhancing system capacity.

To validate the superiority of the proposed ChannelMamba model, its performance is compared against the following representative and SOTA channel prediction baseline models:

- **PAD:** [43] The prony angular-delay (PAD) method estimates and extrapolates channel parameters in the angle-delay domain by leveraging the sparsity of physical models.
- **LSTM:** [24] LSTM networks employ gated units to effectively capture temporal correlations within the channel sequence.
- **STNN:** [44] The spatio-temporal neural network (STNN) employs a convolutional recurrent architecture to capture spatio-temporal correlations and addresses non-stationarity through differencing and attention mechanisms.

TABLE I
TRAINING HYPERPARAMETERS FOR CHANNELMAMBA

Parameter	Value
Learning Rate	0.0001
Optimizer	AdamW [46]
Learning Rate Scheduler	CosineAnnealingLR [47]
Epochs	500
Batch Size	128
Gradient Clipping	1.0
Dropout	0.1
Hidden dimension (d_{model})	512
Conv dimension (d_{conv})	4
Transformer dimension (d_{tr})	128
Transformer layer (N_{Enc})	1
TD Mamba layer (M_{TD})	2
BiMamba layer (M_{Bi})	3

- **Transformer:** [11] The Transformer model utilizes self-attention to evaluate historical information and predict future channels simultaneously, effectively modeling global dependencies while preventing sequential error propagation.
- **Linformer:** [45] The Linformer architecture utilizes linear attention mechanisms with $O(N)$ complexity to efficiently process long sequences for time-aware MIMO channel prediction, thereby addressing the quadratic complexity bottleneck associated with standard attention.
- **Channelformer:** [12] Channelformer is an attention-based neural network solution specifically designed for wireless channel estimation and prediction. It combines an attention encoder with additional structures, such as convolutional decoders.
- **Naive Prediction:** Naive Prediction forecasts future steps based on the most recently observed channel state, serving as a straightforward extrapolation baseline.

To ensure fairness, all comparative experiments are conducted with identical hardware and environmental conditions. Furthermore, the hyperparameters for each method are optimized using the same tuning strategy to compare them at their respective best performance levels. The key hyperparameters are summarized in Table I.

B. Performance Evaluation

1) **Robustness Evaluation:** To comprehensively assess the performance stability of the model with complex and dynamic channel conditions, this subsection investigates its predictive capabilities across varying SNRs and UE mobility speeds. The focus is on analyzing its robustness against noise interference and rapid channel variations.

To evaluate the stability of prediction accuracy with varying noise levels, experiments are conducted with the UE mobility speed fixed at 60 km/h. A noise term, denoted as $\mathbf{W}[t]$, is added to the historical CSI data, adjusting the noise power to ensure that the input SNR of the sequence varies from 0 dB to 30 dB. The NMSE for predicting future CSI is recorded and compared for the ChannelMamba model and all baseline models with both time division duplex (TDD) and frequency division duplex (FDD) systems. From Fig. 4 and Fig. 5, we know that

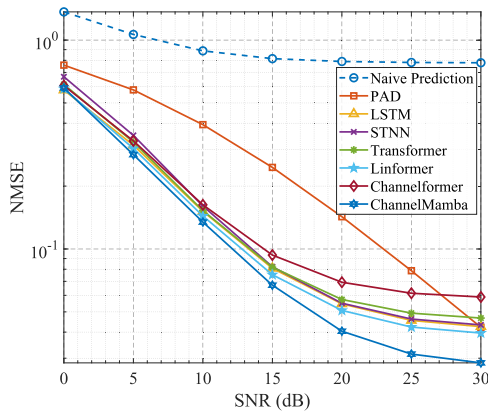


Fig. 4. Comparisons on NMSE performance for different SNR values with TDD systems.

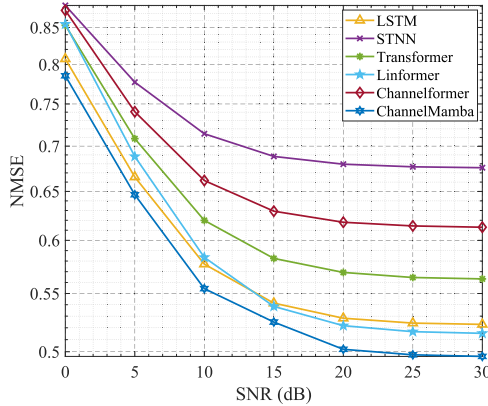


Fig. 5. Comparisons on NMSE performance for different SNR values with FDD systems.

- In both TDD and FDD systems, NMSE decreases as the SNR increases for all models, indicating a reduction in the impact of noise. However, the NMSE for FDD systems remains consistently higher due to the inherent complexity associated with cross-band prediction. The Naive Prediction baseline is omitted from the FDD plots for visual clarity, as its high NMSE values would compress the y-axis and obscure the performance differences among the other models.
- ChannelMamba consistently achieves the lowest NMSE across the 0-30 dB SNR range in both TDD and FDD systems, demonstrating superior accuracy under varying noise conditions. This advantage arises from the selective state-space mechanism of the Mamba core, which effectively captures channel dynamics while mitigating noise propagation through selective updates.
- The consistently low NMSE of ChannelMamba across various SNRs underscores its robustness against noise when compared to baseline models. This advantage is further enhanced by parameter sharing, which improves generalization, and by dual-domain input fusion, which provides complementary information that is essential for FDD systems.

To investigate the predictive capability of the model with highly time-varying channel conditions, we examine the impact of UE mobility. In this experiment, the UE mobility speed v is progressively increased from 10 km/h to 100 km/h,

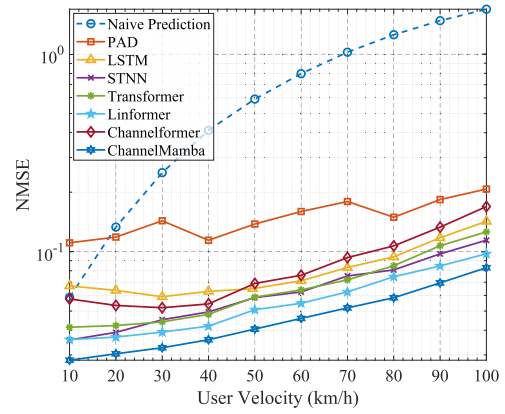


Fig. 6. Comparisons on NMSE performance for different values of UE velocity with TDD systems.

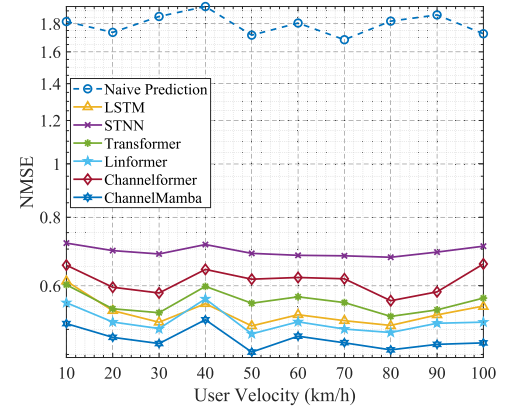


Fig. 7. Comparisons on NMSE performance for different values of UE velocity with FDD systems.

simulating varying rates of channel change. The NMSE for predicting future CSI is recorded and compared for the ChannelMamba model and all baseline models with both TDD and FDD systems. From Fig. 6 and Fig. 7, we know that

- In both TDD and FDD systems, NMSE increases with UE velocity across all models, with FDD levels generally being higher. This trend reflects the increasing challenge of tracking rapid channel dynamics, particularly for cross-band prediction in mobile FDD systems. While minor fluctuations occur due to statistical variations in the test set, the overall trend of performance degradation with increasing velocity is evident across all models.
- ChannelMamba consistently demonstrates the lowest NMSE and the slowest increase in NMSE with velocity in both scenarios, with its advantage becoming more pronounced at higher speeds ($v > 60$ km/h). This superior performance in high-mobility situations is attributed to the ability of Mamba modules to efficiently model long-term dependencies and rapid channel variations.
- The robustness of ChannelMamba against mobility effects is evident across various velocities when compared to baseline models. This strength arises from its architecture, where Mamba modules effectively manage evolving cross-dimensional dependencies. Additionally, the dual-path input enhances adaptability, particularly in mobile FDD channels.

TABLE II
COMPARISONS ON SE PERFORMANCE (Bps/Hz)

Duplex mode	Naive prediction	LSTM	STNN	Transformer	Linformer	Channelformer	ChannelMamba
TDD	5.3078	6.7915	6.2669	6.3430	6.7136	6.3177	6.8087
FDD	5.0021	5.9449	5.0326	5.8492	6.0153	5.6450	6.0878

TABLE III
GENERALIZATION PERFORMANCE COMPARISONS WITH NMSE AND TRANSFER GAP

Scenario	Performance Type	LSTM	STNN	Transformer	Linformer	Channelformer	ChannelMamba
UMi-LOS	Native Perf.	0.1785	0.2191	0.2107	0.1893	0.2194	0.1652
	Transfer Perf.	0.2256	0.2657	0.2562	0.2240	0.2646	0.1917
	GAP (%)	26.4%	21.3%	21.6%	18.3%	20.6%	16.0%
UMi-NLOS	Native Perf.	0.0593	0.0668	0.0651	0.0582	0.0725	0.0521
	Transfer Perf.	0.0681	0.0742	0.0731	0.0659	0.0817	0.0576
	GAP (%)	14.8%	11.1%	12.3%	13.2%	12.7%	10.6%

Note: GAP (%) quantifies the percentage increase in NMSE from Native to Transfer performance. A lower GAP indicates stronger generalization.

To further evaluate the communication effectiveness and reliability of these models, this paper records and compares the SE metric for each method in both TDD and FDD systems. From Table II, we know that

- SE values are below the ideal range for all methods, with FDD SE typically lower than TDD SE. This discrepancy reflects the impact of prediction inaccuracies on achievable rates, underscoring the greater challenges posed by cross-band prediction in FDD systems.
- ChannelMamba consistently achieves the highest SE among the evaluated models in both TDD and FDD systems. This superior SE is a direct result of its enhanced prediction accuracy, which is facilitated by the Mamba-based architecture that effectively captures complex spatio-temporal-frequency channel dynamics.

2) *Generalization Performance Evaluation:* To rigorously evaluate the adaptability of the proposed ChannelMamba model, we assess its performance when trained on the UMa-NLOS scenario and tested on two previously unseen UMi-NLOS and UMi-LOS scenarios. Additionally, we establish a native performance baseline for each model to quantify the performance degradation caused by this domain shift, a metric we term transfer GAP. From Fig. 8 ~ Fig. 15 and Table III, we know that

- Across all test conditions, ChannelMamba consistently outperforms baseline models. Notably, although all models experience performance degradation on the unseen UMi-NLOS data, the performance advantage of ChannelMamba becomes more pronounced, indicating superior generalization. This is evidenced by its achievement of the lowest NMSE in both native and transfer settings, as well as the smallest transfer GAP, with values of 10.6% for UMi-NLOS and 16.0% for UMi-LOS (Table III). This enhanced generalization arises from its architecture,

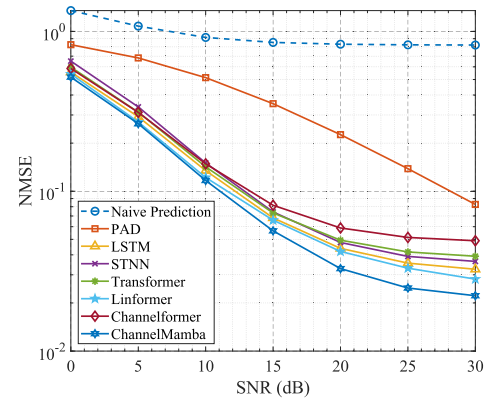


Fig. 8. Comparison of NMSE performance for different SNR values in the UMi-NLoS scenario with TDD systems.

which integrates dual-domain inputs to capture fundamental physical dynamics and employs parameter sharing to regularize the model and reduce overfitting.

- The absolute NMSE values are notably higher in the UMi-LOS scenario. This is attributed to the intrinsic properties of LoS channels, where the prediction task is highly sensitive to phase errors of the single dominant path. Even minor inaccuracies in its phase can disproportionately increase the NMSE, making the prediction task more challenging than in richly scattered NLOS environments.
- Crucially, it is in this more distinct and sensitive UMi-LOS environment that the performance advantage of ChannelMamba is most pronounced. Its architectural design enables it to better capture the critical dynamics of the dominant path, leading to a more significant performance margin over baselines and confirming its advanced modeling capabilities in diverse propagation conditions.

TABLE IV
EFFICIENCY COMPARISONS

Metric	PAD	LSTM	STNN	Transformer	Linformer	Channelformer	ChannelMamba
Network parameters (M)	—	5.68	0.02	7.50	6.41	6.87	4.65
FLOPs (M)	—	87.654	7.870	77.652	102.662	76.151	45.103
Inference time (ms)	41.02	1.08	34.99	9.24	4.16	4.79	3.93

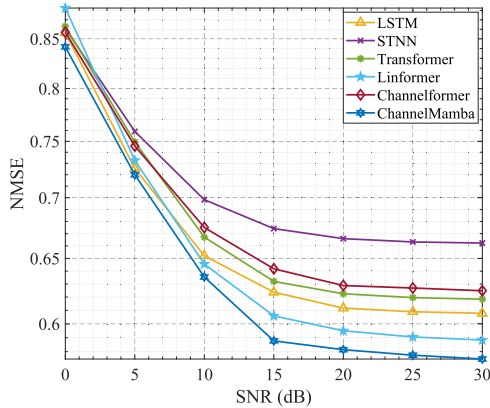


Fig. 9. Comparison of NMSE performance for different SNR values in the UMi-NLoS scenario with FDD systems.

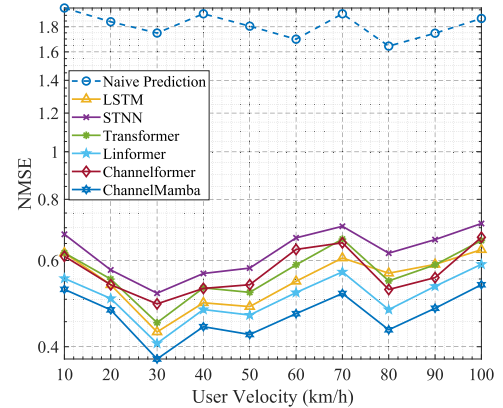


Fig. 11. Comparison of NMSE performance for different values of UE velocity in the UMi-NLoS scenario with FDD systems.

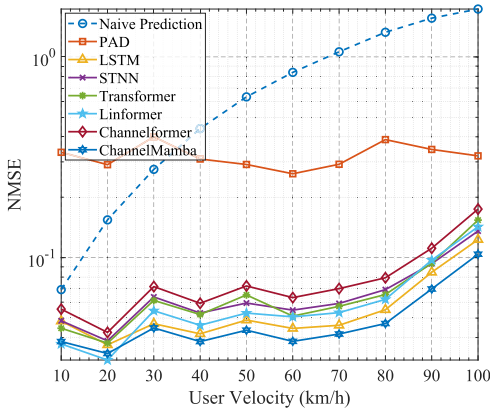


Fig. 10. Comparison of NMSE performance for different values of UE velocity in the UMi-NLoS scenario with TDD systems.

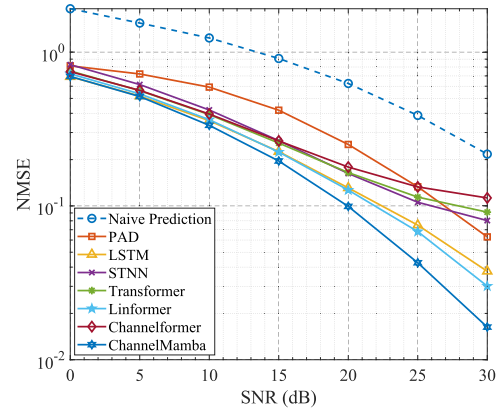


Fig. 12. Comparison of NMSE performance for different SNR values in the UMi-LoS scenario with TDD systems.

3) *Efficiency Evaluation*: This subsection evaluates the performance of the ChannelMamba model and key baseline models in relation to network parameters, floating point operations (FLOPs) and inference time. We utilize a fixed input history length of $P = 16$. From Table IV, we know that

- ChannelMamba achieves a competitive model size (in terms of parameters) while requiring significantly fewer FLOPs compared to attention-based models such as the Transformer and Channelformer. This reduction in FLOPs is a direct result of the linear complexity scaling of the selective state-space mechanism of the Mamba architecture, in contrast to the quadratic scaling $O(N^2)$ of standard self-attention.
- In practical execution, ChannelMamba exhibits a significantly lower inference time compared to the standard

Transformer and PAD methods. This observed speedup underscores the efficiency of the selective scan operation within the Mamba blocks during inference, which circumvents the extensive matrix computations associated with global attention. As a result, this translates the theoretical reduction in FLOPs reduction into substantial latency gains, making it suitable for real-time predictions.

4) *Ablation Study*: To validate the effectiveness of the key design components within the ChannelMamba model and their contributions to overall performance, this subsection presents a series of ablation experiments. Several variants of the ChannelMamba model are constructed by systematically removing specific modules or modifying key strategies. The performance of these variants, measured in terms of average NMSE and SE across all tested velocities ranging from 10 km/h to 100 km/h,

TABLE V
ABLATION STUDY RESULTS

Metric	ChannelMamba	w/o Freq. path	w/o Delay path	w/o Shared weights	w/o Attention	w/o BiMamba
NMSE	0.0548	0.0672	0.0691	0.0625	0.0671	0.0741
Network parameters (M)	4.65	4.13	4.13	5.79	4.27	4.65
FLOPs (M)	45.103	38.539	38.539	64.523	41.857	33.605
Inference time (ms)	3.93	3.81	3.81	6.08	3.70	3.66

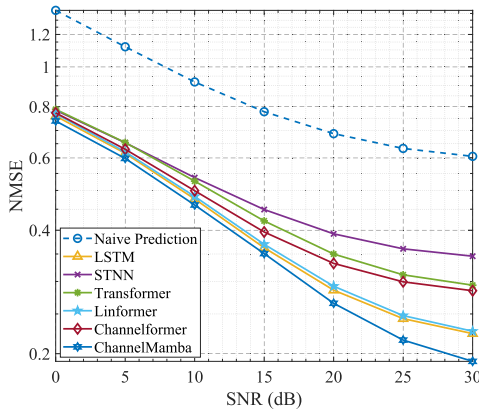


Fig. 13. Comparison of NMSE performance for different SNR values in the UMi-LoS scenario with FDD systems.

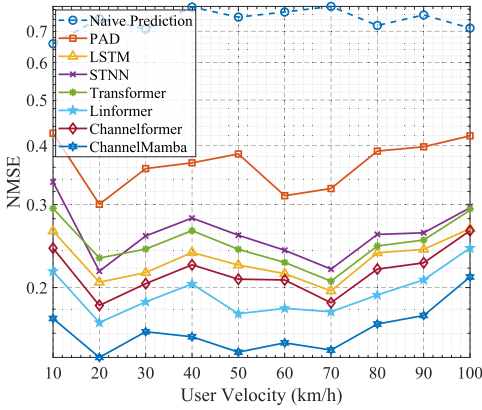


Fig. 14. Comparison of NMSE performance for different values of UE velocity in the UMi-LoS scenario with TDD systems.

is compared against the complete model the TDD systems. From Table V, we know that

- Ablation studies confirm the effectiveness of the synergistic design, as removing any single component leads to a significant decline in performance. Specifically, omitting the dual-path inputs underscores the necessity of complementary channel representation. Furthermore, the performance degradation caused by removing the BiMamba block or replacing the lightweight attention module with a simple projection block highlights their distinct roles in capturing essential channel dependencies and providing critical final refinement.
- The limited performance improvement observed from disabling parameter sharing, despite a substantial increase in

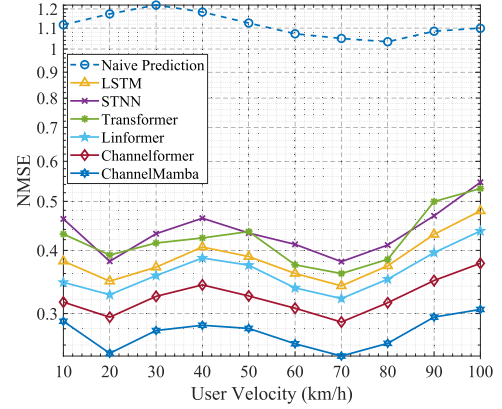


Fig. 15. Comparison of NMSE performance for different values of UE velocity in the UMi-LoS scenario with FDD systems.

model size, validates the efficiency of the weight-sharing strategy. This result suggests that parameter sharing offers a strong regularizing effect, enhancing model efficiency without compromising predictive accuracy.

5) *Effectiveness of Mamba as a Backbone*: To further investigate the potential of the Mamba block as a core sequence processing engine in comparison to Transformer-based approaches, and to understand the interplay between Mamba and attention mechanisms within ChannelMamba, we conduct a backbone replacement experiment. This experiment compares the core processing capabilities of Mamba blocks with those of Transformer components within established attention-utilizing frameworks, isolating the effects of other design choices in ChannelMamba.

We select Transformer [11] and Channelformer [12] as baseline frameworks. In each case, the peripheral modules remain unchanged, while the multi-head attention blocks are replaced with stacks of Mamba blocks. We then compare the NMSE, inference time and FLOPs of the original baselines against their Mamba-backbone variants. From Table VI, we know that

- Both the Transformer and Channelformer frameworks demonstrate enhanced prediction accuracy (indicated by a lower NMSE) when their core multi-head attention blocks are replaced with Mamba blocks. This result suggests that the selective state-space mechanism of the Mamba block effectively captures essential channel dynamics, potentially outperforming standard attention mechanisms for this task within these architectures.
- Simultaneously, this substitution of the backbone leads to a significant reduction in both FLOPs and inference

TABLE VI
COMPARISONS ON BACKBONE NETWORK PERFORMANCE

Baseline model	Backbone	Network parameters (M)	FLOPs (M)	Inference time (ms)	NMSE	SE (bps/Hz)
Transformer [11]	Multi-head attention	7.50	77.652	9.24	0.0640	6.927
	Mamba block	4.04	4.343	4.80	0.0609	6.941
Channelformer [12]	Multi-head attention	6.87	76.151	4.79	0.0759	6.891
	Mamba block	3.96	8.976	3.24	0.0673	6.944

time compared to the original attention cores. This simultaneous improvement in efficiency and accuracy is significant. The reduction in FLOPs arises directly from substituting the quadratic-complexity ($O(N^2)$) self-attention mechanism with the linear-time ($O(N)$) SSM of Mamba. The superior NMSE indicates that the selective state update mechanism of Mamba, which dynamically modulates information flow based on input, is inherently more effective at capturing the non-stationary dynamics of wireless channels than the all-pairs interaction characteristic of self-attention.

VII. CONCLUSION

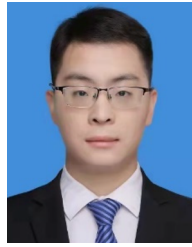
This paper has presented ChannelMamba, a novel end-to-end MIMO-OFDM channel prediction model that innovatively utilizes the Mamba architecture. By leveraging the linear complexity and robust sequence modeling capabilities of the selective SSM of the Mamba architecture, ChannelMamba aims to address the efficiency bottlenecks associated with conventional deep learning methods, such as Transformers. The model incorporates a dual-domain input module and subsequently employs a core processing stage that includes parameter-sharing temporal Mamba modules for efficient handling of temporal dependencies, a bidirectional Mamba structure for in-depth modeling of cross-dimensional feature interactions, and a lightweight attention mechanism to enhance global context awareness. Finally, extensive experimental evaluations have demonstrated that ChannelMamba significantly outperforms baseline methods in terms of prediction accuracy, robustness, generalization and computational efficiency.

In future work, the ChannelMamba model will be validated using real-world measurement datasets. Although extensive simulations have demonstrated its effectiveness in standardized environments, this step is crucial for evaluating its robustness against practical channel impairments.

REFERENCES

- [1] J. Park, F. Sohrabi, A. Ghosh, and J. G. Andrews, "End-to-end deep learning for TDD MIMO systems in the 6G upper midbands," *IEEE Trans. Wireless Commun.*, vol. 24, no. 3, pp. 2110–2125, Mar. 2025.
- [2] X. Hu, Y. Huo, X. Dong, F.-Y. Wu, and A. Huang, "Channel prediction using adaptive bidirectional GRU for underwater MIMO communications," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 3250–3263, Jan. 2024.
- [3] H. Chang, C.-X. Wang, R. Feng, C. Huang, L. Hou, and E.-H.-M. Aggoune, "Beam domain channel modeling and prediction for UAV communications," *IEEE Trans. Wireless Commun.*, vol. 24, no. 2, pp. 969–983, Feb. 2025.
- [4] F. Peng, S. Zhang, W. Chen, and Q. Wu, "Beamforming oriented angular domain channel prediction for mixed LOS and non-LOS SIMO environments with high mobility," *IEEE Trans. Wireless Commun.*, vol. 23, no. 5, pp. 4315–4330, May 2024.
- [5] C. Wu, X. Yi, Y. Zhu, W. Wang, L. You, and X. Gao, "Channel prediction in high-mobility massive MIMO: From spatio-temporal autoregression to deep learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1915–1930, Jul. 2021.
- [6] Y. Wei, M.-M. Zhao, A. Liu, and M.-J. Zhao, "Channel tracking and prediction for IRS-aided wireless communications," *IEEE Trans. Wireless Commun.*, vol. 22, no. 1, pp. 563–579, Jan. 2023.
- [7] D. Ying and F. Ye, "Deep learning supported path prediction and channel estimation for MIMO-OTFS system with high delay resolution," *IEEE Trans. Veh. Technol.*, vol. 74, no. 3, pp. 3584–3597, Mar. 2025.
- [8] T. Zhou et al., "Transformer network based channel prediction for CSI feedback enhancement in AI-native air interface," *IEEE Trans. Wireless Commun.*, vol. 23, no. 9, pp. 11154–11167, Sep. 2024.
- [9] S. Zhang, S. Zhang, Y. Mao, L. K. Yeung, B. Clerckx, and T. Q. S. Quek, "Transformer-based channel prediction for rate-splitting multiple access-enabled vehicle-to-everything communication," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 12717–12730, Oct. 2024.
- [10] S. Zhang, S. Zhang, W. Yuan, and T. Q. S. Quek, "Transformer-empowered predictive beamforming for rate-splitting multiple access in non-terrestrial networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 19776–19788, Dec. 2024.
- [11] H. Jiang, M. Cui, D. W. K. Ng, and L. Dai, "Accurate channel prediction based on transformer: Making mobility negligible," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 9, pp. 2717–2732, Sep. 2022.
- [12] D. Luan and J. Thompson, "Channelformer: Attention based neural solution for wireless channel estimation and effective online training," *IEEE Trans. Wireless Commun.*, vol. 22, no. 10, pp. 6562–6577, Oct. 2023.
- [13] A. Hameed, J. Violos, N. Santi, A. Leivadreas, and N. Mitton, "FeD-TST: Federated temporal sparse transformers for QoS prediction in dynamic IoT networks," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 2, pp. 1055–1069, Apr. 2025.
- [14] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.
- [15] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024, pp. 1–52.
- [16] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," 2021, *arXiv:2111.00396*.
- [17] Y. Yang, Z. Xing, L. Yu, C. Huang, H. Fu, and L. Zhu, "Vivim: A video vision mamba for medical video segmentation," 2024, *arXiv:2401.14168*.
- [18] Y. Liu et al., "VMamba: Visual state space model," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 103031–103063.
- [19] A. Hatamizadeh and J. Kautz, "MambaVision: A hybrid mamba-transformer vision backbone," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 25261–25270.
- [20] X. Xu, Y. Liang, B. Huang, Z. Lan, and K. Shu, "Integrating mamba and transformer for long-short range time series forecasting," 2024, *arXiv:2404.14757*.
- [21] B. N. Patro and V. S. Agneeswaran, "SiMBA-TS: Simplified channel mixing and mamba for long-term time series forecasting," in *Proc. ICASSP - IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2025, pp. 1–5.
- [22] M. A. Ahamed and Q. Cheng, "TSCMamba: Mamba meets multi-view learning for time series classification," *Inf. Fusion*, vol. 120, Aug. 2025, Art. no. 103079.

- [23] T. Gong et al., "Holographic MIMO communications with arbitrary surface placements: Near-field LoS channel model and capacity limit," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 6, pp. 1549–1566, Jun. 2024.
- [24] W. Jiang and H. D. Schotten, "Deep learning for fading channel prediction," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 320–332, 2020.
- [25] C. Huang, C.-X. Wang, Z. Li, Z. Qian, J. Li, and Y. Miao, "A frequency domain predictive channel model for 6G wireless MIMO communications based on deep learning," *IEEE Trans. Commun.*, vol. 72, no. 8, pp. 4887–4902, Aug. 2024.
- [26] S. Mourya, P. Reddy, S. Amuru, and K. K. Kuchi, "Spectral temporal graph neural network for massive MIMO CSI prediction," *IEEE Wireless Commun. Lett.*, vol. 13, no. 5, pp. 1399–1403, May 2024.
- [27] S. Fan, H. Li, X. Liang, Z. Liu, X. Gu, and L. Zhang, "E2ENet: An end-to-end channel prediction neural network based on uplink pilot for FDD systems," *IEEE Wireless Commun. Lett.*, vol. 13, no. 5, pp. 1285–1289, May 2024.
- [28] B. Liu, X. Liu, S. Gao, X. Cheng, and L. Yang, "LLM4CP: Adapting large language models for channel prediction," *J. Commun. Inf. Netw.*, vol. 9, no. 2, pp. 113–125, Jun. 2024.
- [29] S. Kim, A. Lee, H. Ju, K. A. Ngo, J. Moon, and B. Shim, "Transformer-based channel parameter acquisition for terahertz ultra-massive MIMO systems," *IEEE Trans. Veh. Technol.*, vol. 72, no. 11, pp. 15127–15132, Nov. 2023.
- [30] K. Jin, X. Ren, J. Huang, and H. Shi, "Collaborative cloud-edge channel prediction for dynamic industrial wireless networks: An obstacle-aware channel transformative architecture," in *Proc. 10th Int. Conf. Comput. Commun. (ICCC)*, Dec. 2024, pp. 2146–2152.
- [31] B. Liu, S. Gao, X. Liu, X. Cheng, and L. Yang, "WiFo: Wireless foundation model for channel prediction," *Sci. China Inf. Sci.*, vol. 68, May 2025, Art. no. 162302. [Online]. Available: <http://www.sciengine.com/publisher/ScienceChinaPress/journal/SCIENECCHINAInformationSciences///10.1007/s11432-025-4349-0>
- [32] R. Bhirangi et al., "Hierarchical state space models for continuous sequence-to-sequence modeling," 2024, *arXiv:2402.10211*.
- [33] X. Jiang, C. Han, and N. Mesgarani, "Dual-path mamba: Short and long-term bidirectional selective structured state space models for speech separation," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2025, pp. 1–5.
- [34] W. Yu and X. Wang, "MambaOut: Do we really need mamba for vision?," 2024, *arXiv:2405.07992*.
- [35] Z. Wang et al., "Is mamba effective for time series forecasting?," *Neurocomputing*, vol. 619, Feb. 2025, Art. no. 129178.
- [36] Q. Li, J. Qin, D. Cui, D. Sun, and D. Wang, "CMMamba: Channel mixing mamba for time series forecasting," *J. Big Data*, vol. 11, no. 1, p. 153, Oct. 2024.
- [37] H. Ma et al., "A mamba foundation model for time series forecasting," 2024, *arXiv:2411.02941*.
- [38] Q. Zhu, C.-X. Wang, B. Hua, K. Mao, S. Jiang, and M. Yao, "3GPP TR 38.901 channel model," in *The Wiley 5G Ref: The Essential 5G Reference Online*. Hoboken, NJ, USA: Wiley, 2021, pp. 1–35.
- [39] T. Jiang et al., "3GPP standardized 5G channel model for IIoT scenarios: A survey," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8799–8815, Jun. 2021.
- [40] Z. Shao, X. Yao, Z. Wang, and J. Gao, "ST-MambaSync: The complement of mamba and transformers for spatial-temporal in traffic flow prediction," 2024, *arXiv:2404.15899*.
- [41] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1474–1487.
- [42] S. Jaeckel, L. Raschkowski, K. Börner, and L. Thiele, "QuaDRiGa: A 3-D multi-cell channel model with time evolution for enabling virtual field trials," *IEEE Trans. Antennas Propag.*, vol. 62, no. 6, pp. 3242–3256, Jun. 2014.
- [43] H. Yin, H. Wang, Y. Liu, and D. Gesbert, "Addressing the curse of mobility in massive MIMO with prony-based angular-delay domain channel predictions," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 12, pp. 2903–2917, Dec. 2020.
- [44] G. Liu, Z. Hu, L. Wang, J. Xue, H. Yin, and D. Gesbert, "Spatio-temporal neural network for channel prediction in massive MIMO-OFDM systems," *IEEE Trans. Commun.*, vol. 70, no. 12, pp. 8003–8016, Dec. 2022.
- [45] Y. Jin, Y. Wu, Y. Gao, S. Zhang, S. Xu, and C.-X. Wang, "LinFormer: A linear-based lightweight transformer architecture for time-aware MIMO channel prediction," 2024, *arXiv:2410.21351*.
- [46] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–19.
- [47] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–16.



Huaguang Shi received the B.S. degree in electronic science and technology from Zhengzhou University, Zhengzhou, China, in 2014, and the Ph.D. degree in measurement technique and automation equipment from the University of Chinese Academy of Sciences, Beijing, China, in 2021. He is currently an Associate Professor with the School of Artificial Intelligence, Henan University, Zhengzhou. His current research interests include the Industrial Internet of Things, wireless networks, and multi-agent learning.



Kaibo Jin is currently pursuing the bachelor's degree with Henan University, Zhengzhou, Henan. His research interests include deep learning and industrial wireless networks.



Xiaoquan Ren is currently pursuing the bachelor's degree with Henan University, Zhengzhou, Henan. Her research interests include industrial wireless networks and deep learning.



Wei Li received the Ph.D. degree in cartography and geographic information systems from Henan University, China, in 2022. She is currently an Associate Professor at the School of Artificial Intelligence, Henan University. Her research interests include cooperative control of multi-agent systems, vehicular edge computing, and intelligent transportation systems.



Yi Zhou (Member, IEEE) received the B.S. degree in electronic engineering from the First Aeronautic Institute of Air Force, China, in 2002, and the Ph.D. degree in control systems and theory from Tongji University, China, in 2011. He is currently a Full Professor and the Deputy Dean of the School of Artificial Intelligence, Henan University, China. He is also the Director of the International Joint Research Laboratory for Cooperative Vehicular Networks, Henan, China. His research interests include vehicular cyber-physical systems and multi-agent

collaboration.